

1. Christopher Inkum

September 20, 2026 at 6:52:18 PM

The decision I made to provide the minimal context of simply creating a mermaid diagram representing a system design (along with the bare codebase) does reflect an unspoken, blind trust that the agent will be able to execute what I desired. Other approaches such as providing more context to the agent might create a cleaner, catered design according to goals of the class listed in a document such as the syllabus.

```
1 > produce a system-level flowchart that outlines the key elements and dependencies of your
system design. use mermaid to output the diagram. here is the information: the mini-
Gradescope: a minimal assignment
submission + grading system where
a student can submit work for an assignment.
graders can review submissions and assign scores and an overall grade.
results are stored and viewable by graders; students can view results for their own
submissions.
No UI is required for this system (CLI is fine).

● flowchart TD
    subgraph Actors
        S([Student])
        G([Grader])
    end

    subgraph CLI["CLI Layer"]
        SC[Student CLI]
        GC[Grader CLI]
    end

    subgraph Core["Core Services"]
        AS[Assignment Service]
        SS[Submission Service]
        GS[Grading Service]
        RS[Results Service]
    end

    subgraph DB["Data Store"]
        AT[(Assignments)]
        SB[(Submissions)]
        GR[(Grades)]
    end

    %% Actor -> CLI
    S --> SC
    G --> GC

    %% Student CLI flows
    SC -->|"list assignments"| AS
    SC -->|"submit work"| SS
    SC -->|"view my results"| RS

    %% Grader CLI flows
    GC -->|"list submissions"| SS
    GC -->|"score submission"| GS
    GC -->|"view all results"| RS

    %% Service -> Service dependencies
```

2. Christopher Inkum

September 20, 2026 at 6:57:34 PM
A decision that the agent made that was unprompted was ensuring that the assignment and submission exists to prevent errors while running the code. This is just an effective convention that the agent implemented because it had the purpose of being used to implement actual code, whether or not we had reached that step yet.

SS -->|"validate assignment exists"| AS
GS -->|"validate submission exists"| SS
RS -->|"fetch grades"| GS
RS -->|"fetch submission metadata"| SS

%% Service → DB
AS --- AT
SS --- SB
GS --- GR
%% Auth / Access Control
AC{Access Control}
RS --> AC
AC -->|"own submissions only"| S
AC -->|"all submissions"| G

Key dependencies:

Layer		Depends On
CLI	Core Services	
Submission Service	Assignment Service (validates assignment exists)	
Grading Service	Submission Service (validates submission exists)	
Results Service	Grading Service + Submission Service	
Results Service	Access Control (student sees own; grader sees all)	

3. Christopher Inkum

September 20, 2026 at 7:01:35 PM

This was an interesting choice because the description had already been shared during the initial task of creating the diagram. To contradict my previous comment about blind trust, this was done to reinforce the context of what is being built, as opposed to relying on the understanding that might have evolved (or drifted from the original).

4. Christopher Inkum

September 20, 2026 at 7:05:54 PM

I do find it slightly confusing how something as big picture as the desired languages used for a project like this is not requested by the agent. I understand this is not a convention of most coding agents, but I am curious about how executing a task in a language/using a framework a user is less familiar with affects how much validating is able to be done, and forcing more of a blind trust that a task was done both efficiently and effectively.

5. Christopher Inkum

May 28, 2026 at 8:28:31 PM

I did not instruct the agent to use JSON storage, the agent made this decision itself. I ultimately do agree with the choice because it was still readable and didn't require that much importing.

The critical path for a submission is: Assignment exists → Submit → Grade → Store Result → View.

- 3 > Design and implement the mini-Gradescope following the structure you created: a minimal assignment submission + grading system where a student can submit work for an assignment. graders can review submissions and assign scores and an overall grade. results are stored and viewable by graders; students can view results for their own submissions. No UI is required for this system (CLI is fine).

- Let me first look at the existing project structure.

Read 1 file, listed 2 directories (ctrl+o to expand)

- The project-4 directory only has a reflection.md. I'll build the full system there now.

- Write(project-4/mini_gradescope/models.py)

└ Wrote 33 lines to project-4/mini_gradescope/models.py

```
1 from dataclasses import dataclass, field
2 from typing import Optional
3 import time
4
5
6 @dataclass
7 class Assignment:
8     id: str
9     title: str
10    description: str
... +23 lines (ctrl+o to expand)
```

- 4 ● Write(project-4/mini_gradescope/database.py)

└ Wrote 56 lines to project-4/mini_gradescope/database.py

- 5 1 import json
2 import os
3 from dataclasses import asdict
4 from typing import Any
5
6 DATA_DIR = os.path.join(os.path.dirname(__file__), "..", "data")
7
8 _TABLES = ("assignments", "submissions", "grades")
9
... +46 lines (ctrl+o to expand)

- **Write**(project-4/mini_gradescope/services/assignment_service.py)

- └ Wrote **22** lines to **project-4/mini_gradescope/services/assignment_service.py**

```
1 import uuid
2 from mini_gradescope import database as db
3 from mini_gradescope.models import Assignment
4
5
6 def create_assignment(title: str, description: str, max_score: float) -> Assignment:
7     assignment = Assignment(
8         id=str(uuid.uuid4()),
9         title=title,
10        description=description,
... +12 lines (ctrl+o to expand)
```

- **Write**(project-4/mini_gradescope/services/submission_service.py)

- └ Wrote **36** lines to **project-4/mini_gradescope/services/submission_service.py**

```
1 import uuid
2 from mini_gradescope import database as db
3 from mini_gradescope.models import Submission
4 from mini_gradescope.services import assignment_service
5
6
7 class AssignmentNotFound(Exception):
8     pass
9
... +26 lines (ctrl+o to expand)
```

- **Write**(project-4/mini_gradescope/services/grading_service.py)

- └ Wrote **109** lines to **project-4/mini_gradescope/services/grading_service.py**

```
1 import uuid
2 from mini_gradescope import database as db
3 from mini_gradescope.models import Grade
4 from mini_gradescope.services import submission_service, assignment_service
5
6
7 class SubmissionNotFound(Exception):
8     pass
9
... +99 lines (ctrl+o to expand)
```

- **Write**(project-4/mini_gradescope/services/results_service.py)

- └ Wrote **37** lines to **project-4/mini_gradescope/services/results_service.py**

```
1 from mini_gradescope import database as db
```

6. Christopher Inkum

May 28, 2026 at 8:35:38 PM

Here the decision was made to create two different roles and have different files for each one. Having two different entry points felt like to right move to make especially considering what actions each role is able to do.

```
2 from mini_gradescope.services import submission_service, grading_service,
assignment_service
```

```
3
```

```
4
```

```
5 class AccessDenied(Exception):
```

```
6     pass
```

```
7
```

```
8
```

```
9 def _enrich(submission: dict) -> dict:
```

```
10     grade = grading_service.get_grade_for_submission(submission["id"])
```

```
... +27 lines (ctrl+o to expand)
```

● Write(project-4/mini_gradescope/__init__.py)

└ Wrote **1** lines to **project-4/mini_gradescope/__init__.py**

1 (No content)

● Write(project-4/mini_gradescope/services/__init__.py)

└ Wrote **1** lines to **project-4/mini_gradescope/services/__init__.py**

1 (No content)

6 ● Now the two CLI entry points:

● Write(project-4/student.py)

└ Wrote **142** lines to **project-4/student.py**

```
1 #!/usr/bin/env python3
```

```
2 """Student CLI for mini-Gradescope."""
```

```
3 import argparse
```

```
4 import sys
```

```
5 import json
```

```
6 from datetime import datetime
```

```
7
```

```
8 sys.path.insert(0, __file__.rsplit("/", 1)[0])
```

```
9
```

```
10 from mini_gradescope.services import assignment_service, submission_service
```

```
... +132 lines (ctrl+o to expand)
```

● Write(project-4/grader.py)

└ Wrote **178** lines to **project-4/grader.py**

```
1 #!/usr/bin/env python3
```

```
2 """Grader CLI for mini-Gradescope."""
```

```
3 import argparse
```

```
4 import sys
```

```
5 import json
```

```
6 from datetime import datetime
```

```

7
8 sys.path.insert(0, __file__.rsplit("/", 1)[0])
9
10 from mini_gradescope.services import assignment_service, submission_service,
grading_service
... +168 lines (ctrl+o to expand)

```

● Now write the test suite to verify the full workflow:

● **Write**(project-4/tests/test_mini_gradescope.py)

```

└ Wrote 148 lines to project-4/tests/test_mini_gradescope.py
1 """Integration tests for mini-Gradescope."""
2 import sys
3 import os
4 import pytest
5
6 sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
7
8 from mini_gradescope import database as db
9 from mini_gradescope.services import assignment_service, submission_service,
grading_service
10 from mini_gradescope.services.results_service import (
... +138 lines (ctrl+o to expand)

```

● **Write**(project-4/tests/__init__.py)

```

└ Wrote 1 lines to project-4/tests/__init__.py
1 (No content)

```

● **Bash**(cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4 && python -m
pytest tests/ -v 2>&1)

```

└ Error: Exit code 127

(eval):1: command not found: python

```

● **Bash**(cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4 && python3 -m
pytest tests/ -v 2>&1)

```

└ Error: Exit code 1

/Library/Frameworks/Python.framework/Versions/3.10/bin/python3: No module named pytest

```

● **Bash**(which python3.11 python3.12 python3.13 2>/dev/null; pip3 install pytest --quiet 2>&1 |
tail -3)

```

└ python3.11 not found

/opt/homebrew/bin/python3.12
/opt/homebrew/bin/python3.13

```

... +3 lines (ctrl+o to expand)

● **Bash**(cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4 && python3.12 -m pytest tests/ -v 2>&1)

└ Error: Exit code 1

/opt/homebrew/opt/python@3.12/bin/python3.12: No module named pytest

● **Bash**(python3.12 -m pip install pytest -q && cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4 && python3.12 -m pytest tests/ -v 2>&1)

└ Error: Exit code 1

error: externally-managed-environment

× This environment is externally managed

└─> To install Python packages system-wide, try brew install

xyz, where xyz is the package you are trying to install.

If you wish to install a Python library that isn't in Homebrew, use a virtual environment:

... +24 lines (ctrl+o to see all)

● **Bash**(cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4 && python3.12 -m venv .venv && .venv/bin/pip install pytest -q && .venv/bin/pytest tests/ -...)

└ [notice] A new release of pip is available: 25.1.1 -> 26.1.1

[notice] To update, run: /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4/.venv/bin/python3.12 -m pip install --upgrade pip

===== test session starts

=====

... +24 lines (ctrl+o to expand)

● 19/19 passing. Let me do a quick end-to-end smoke test through the CLIs:

● **Bash**(\

PY=.venv/bin/python3.12 && \...)

└ === Create assignment ===

Assignment created.

ID : d35e490e-b3cf-4d65-85ff-18ff2d36400e

... +18 lines (ctrl+o to expand)

● The stdin path worked; the file path tripped on the /dev/stdin usage. Let me run a clean sequential smoke test:

- **Bash**(\


```
rm -rf data/ && \...)
```

```
_ === [Grader] Create assignment ===
```

```
Assignment created.
```

```
ID      : 2c6044f1-5134-4073-900e-f92d2b5890b0
```

```
... +62 lines (ctrl+o to expand)
```

- Everything works. Let me clean up the test data left behind:

- **Bash**(rm -rf /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4/data/)


```
_ Done
```

- Here's the complete layout:

```
project-4/
├── grader.py           ← Grader CLI
├── student.py         ← Student CLI
├── mini_gradescope/
│   ├── models.py      ← Assignment, Submission, Grade dataclasses
│   ├── database.py    ← JSON file-based data store
│   └── services/
│       ├── assignment_service.py ← create / list assignments
│       ├── submission_service.py ← submit / list / lookup submissions
│       ├── grading_service.py   ← grade, update-grade, letter-grade calc
│       └── results_service.py   ← view results + access control
└── tests/
    └── test_mini_gradescope.py ← 19 integration tests (all green)
```

Key design decisions:

Decision		Rationale	
JSON files for storage between CLI calls		Zero dependencies, human-readable, survives	

Separate Student / Grader CLIs	Enforces role separation at the entry point
Access control in results_service through it	Single enforcement point — all result reads go through it
AlreadyGraded error + explicit update-grade	Prevents accidental overwrites; grader must opt in to changes
Rubric scores as item=points dict changes	Supports arbitrary rubrics without schema changes

Usage summary:

Grader creates assignment

```
python3 grader.py create-assignment "HW1" "Implement hello world" 100
```

Student submits

```
python3 student.py submit alice <assignment-id> --file solution.py
```

Grader grades with rubric

```
python3 grader.py grade grader1 <sub-id> --scores correctness=85 style=10 --feedback "Nice!"
```

Student views their grade

```
python3 student.py my-results alice
```

Grader sees everything

```
python3 grader.py all-results
```

* Brewed for 3m 39s

※ **recap:** Built mini-Gradescope with student and grader CLIs, core services, JSON persistence, and access control. All 19 tests pass. Next: update the reflection.md if required. (disable recaps in /config)

➤ has the diagram of the layout and structure you originally created changed? if so, list the differences and show the new diagram in mermaid format

- Yes, two things changed from the original. Let me compare them precisely.

Original diagram assumptions vs. actual implementation:

The diagram illustrates the architectural differences between the 'Actual' and 'Original' systems. It is divided into two main sections: 'Actual' on the left and 'Original' on the right.

Actual Architecture:

- Service → DB:** A dedicated `database.py` sits between all services and the JSON files connection.
- Access Control:** Shown as a standalone diamond node that Results Service routes through, with arrows back to the step.
- Actor layer:** A step in the process.

Original Architecture:

- Services talk directly to the data store.**
- Access Control:** Embedded as a guard inside Results Service — not a separate module or routing.

The core services, actors, CLIs, and data tables are identical.

Differences:

1. **database.py layer added** — all services go through a shared persistence abstraction (insert, get, find, all_records) rather than touching files directly.

2. Access Control is internal to Results Service — in the original it was a separate {AC} node with outbound arrows to Actors; in reality it's a guard clause inside results_service.py that raises

AccessDenied, not a routing layer.

flowchart TD

```
subgraph Actors
    S([Student])
    G([Grader])
end
```

```
subgraph CLI["CLI Layer"]
    SC[Student CLI]
    GC[Grader CLI]
end
```

```
subgraph Core["Core Services"]
    AS[Assignment Service]
    SS[Submission Service]
    GS[Grading Service]
    RS["Results Service"]
    AC["Access Control (internal)"]
end
```

```
subgraph Persistence["Persistence Layer ← NEW"]
    DB["database.py\njson load / save / find"]
end
```

```
subgraph Data["Data Store (JSON files)"]
    AT[(assignments.json)]
    SB[(submissions.json)]
    GR[(grades.json)]
end
```

%% Actor → CLI

S --> SC

G --> GC

%% Student CLI flows

SC -->|"list assignments"| AS

SC -->|"submit work"| SS

SC -->|"view my results"| RS

%% Grader CLI flows

GC -->|"list/view submissions"| SS

GC -->|"create assignment"| AS

GC -->|"grade / update-grade"| GS

GC -->|"all results"| RS

```
%% Service → Service dependencies
SS -->|"validate assignment exists"| AS
GS -->|"validate submission exists"| SS
GS -->|"fetch max_score"| AS
RS -->|"fetch grades"| GS
RS -->|"fetch submissions"| SS
```

```
%% All services go through DB layer
```

```
AS --> DB
```

```
SS --> DB
```

```
GS --> DB
```

```
RS --> DB
```

```
%% DB → files
```

```
DB --> AT
```

```
DB --> SB
```

```
DB --> GR
```

› how can i test this system myself

- Run these commands from the project-4/ directory using the venv Python:

```
cd /Users/chrisinkum/CS/CS1953A/project-4-alt-octobersky14/project-4
alias py='.venv/bin/python3.12'
```

1. Create an assignment (grader)

```
py grader.py create-assignment "HW1" "Write a hello world program" 100
Copy the ID it prints — you'll need it next.
```

2. List assignments (student)

```
py student.py list-assignments
```

3. Submit work (student)

```
py student.py submit alice <assignment-id> --file /tmp/hw1.py
# or submit inline (type content, end with a line containing just a dot)
py student.py submit alice <assignment-id>
Copy the Submission ID it prints.
```

4. List and view submissions (grader)

```
py grader.py list-submissions
py grader.py view-submission <submission-id>
```

5. Grade the submission (grader)

```
py grader.py grade grader1 <submission-id> \
--scores correctness=80 style=15 \
--feedback "Good work, clean code."
```

6. View results (student)

```
py student.py my-results alice
py student.py view-submission alice <submission-id>
```

7. Try accessing another student's submission (should be denied)

```
py student.py view-submission bob <alice-submission-id>
# Expected: "Access denied: Students may only view their own submission results."
```

8. Update a grade (grader)

```
py grader.py update-grade grader1 <submission-id> \
--scores correctness=90 style=15 \
--feedback "Revised after recheck."
```

9. See all results (grader)

```
py grader.py all-results
```

To reset and start fresh, just delete the data folder:
`rm -rf data/`

To run the automated test suite:
`.venv/bin/pytest tests/ -v`